FAST NUCES

Software Engineering

PRISM

Presence Real-time Identification & Security Model

Deliverable #3 - Iteration 2 / Sprint 2

Submitted by:

Areeb Ahmed

Roll No: 23L-0783

Submission Date: 15 April 2026

---

1. Project Introduction

Project Title: PRISM - Presence Real-time Identification & Security Model

Type: Web Application (MERN Stack with decoupled Python AI Service)

1.1 Problem Statement

With the rapid shift toward online and hybrid education systems, verifying genuine human presence has become a major challenge. Traditional attendance methods like passwords, login timestamps, or manual roll calls are highly vulnerable to proxy attendance and identity impersonation. Existing biometric solutions predominantly rely on facial recognition, which creates significant barriers for female students wearing niqab. The system must be web-based, privacy-respecting, and inclusive.

1.2 Proposed Solution

PRISM is a web-based inclusive biometric presence verification system that combines facial recognition and fingerprint authentication. It uses liveness detection, face mismatch checks, and confidence scoring. Students can choose facial, fingerprint, or both methods - ensuring inclusivity. No GPS tracking is involved. Biometric data is stored only as encrypted templates, preserving user privacy.

1.3 Technology Stack

- Frontend: React.js with Tailwind CSS (15 pages fully implemented)
- Backend: Node.js with Express.js (server running, routes defined, MongoDB connected - no main business logic yet)
- Database: MongoDB
- AI Service: Python - facial recognition + fingerprint model code exists, not yet connected via FastAPI to the Node backend
- Deployment: Local (Vercel + Render planned for final delivery)

---

2. Sprint Backlog - Sprint 2

2.1 Module: Biometric Core & Session Management

This sprint focuses on the core biometric features and attendance session handling as defined in the PRISM project proposal. All four user stories selected for Sprint 2 have their frontend UIs fully built. Backend integration and AI service connection are currently in progress.

Sprint Duration: 2 Weeks

Sprint Goal: Working biometric enrollment/verification UI + session management with real-time basic monitoring (frontend complete, backend logic in progress).

2.2 Sprint Backlog Table

Story ID	Priority	User Story Title	Est. Hrs	Status
US-05	High	Biometric Enrollment & Verification UI	10	Done (UI)
US-06	High	Instructor Attendance Session Management	8	Done (UI)
US-07	High	Student Real-time Attendance Marking	7	Done (UI)
US-08	Medium	Live Monitoring Dashboard & Basic Integration	5	In Progress

3. User Stories

US-05 - Biometric Enrollment & Verification UI

Priority: High
Estimated Hours: 10 Hours

User Story:

As a student, I want to enroll and verify my biometric data (face and/or fingerprint) through the web interface so that I can mark attendance securely without proxy risks.

Description:

This covers the complete enrollment flow and verification screen. Users choose face, fingerprint, or both modes. The UI guides them step-by-step with a live webcam feed and confirmation messages. Currently, the frontend enrollment screens are fully implemented in React with Tailwind CSS. The backend storage API and the AI verification call are in progress.

Status Note:

Frontend: Done. Backend: In Progress.

Sub User Stories:

- As a student, I want to select face-only, fingerprint-only, or both enrollment modes.
- As a student, I want a live webcam preview with alignment guides during face enrollment.
- As a student, I want clear success and failure messages after completing enrollment.
- As a student, I want to re-enroll if my previous enrollment is rejected.

Acceptance Criteria:

- Enrollment screen is accessible from the student dashboard.
- Live webcam feed is displayed with alignment guides.
- Student can choose the biometric method before proceeding.
- A confirmation message is displayed upon successful enrollment.
- An error message with retry option is displayed on failure.

US-06 - Instructor Attendance Session Management

Priority: High

Estimated Hours: 8 Hours

User Story:

As an instructor, I want to create and manage time-bound attendance sessions so that students can only mark attendance during the active session window.

Description:

The instructor dashboard allows creating sessions per course with configurable start and end times. A real-time attendance count view is included. All session creation and management UI pages are fully built in React with Tailwind CSS. Backend session logic is currently in progress.

Status Note:

Frontend: Done. Backend: In Progress.

Sub User Stories:

- As an instructor, I want to select a course and set session start and end timings.
- As an instructor, I want to see a live student attendance count during an active session.
- As an instructor, I want to manually close a session before its scheduled end time.
- As an instructor, I want to view a history of past sessions per course.

Acceptance Criteria:

- Session creation form is present and functional in the instructor dashboard.
- An active session displays the live student attendance count.
- Students receive a 'Session Closed' message when accessing outside the time window.
- Instructor can manually end an active session.
- Session history is visible in the instructor panel.

US-07 - Student Real-time Attendance Marking

Priority: High
Estimated Hours: 7 Hours

User Story:

As a student, I want to mark attendance during an active session using my enrolled biometric method so that my presence is verified instantly.

Description:

The student view shows all currently active sessions and allows launching the biometric verification flow with a single click. The UI is fully implemented in React with Tailwind CSS. The backend verification endpoint that calls the Python AI service is currently in progress.

Status Note:

Frontend: Done. Backend: In Progress.

Sub User Stories:

- As a student, I want to see only currently active attendance sessions on my dashboard.
- As a student, I want to start the biometric verification flow with a single click.
- As a student, I want immediate confirmation when my attendance is successfully marked.
- As a student, I want a clear error message if verification fails, with a retry option.

Acceptance Criteria:

- Active sessions are listed on the student dashboard.
- The biometric verification flow launches correctly from the student dashboard.
- A success confirmation is shown after attendance is marked.
- An appropriate error message with retry is shown if verification fails.
- Students cannot mark attendance after the session window has closed.

US-08 - Live Monitoring Dashboard & Basic Integration

Priority: Medium

Estimated Hours: 5 Hours

User Story:

As an instructor, I want a live monitoring dashboard so that I can track attendance progress in real time during an active session.

Description:

The live monitoring dashboard shows pending versus verified students and provides basic periodic updates. The UI is complete and functional in React. Real-time data integration (WebSocket or polling) between the frontend and backend is currently in progress.

Status Note:

Frontend UI: Done. Real-time backend integration: In Progress.

Sub User Stories:

- As an instructor, I want live count updates showing verified and pending students.
- As an instructor, I want low-confidence biometric attempts flagged on the dashboard.
- As an instructor, I want to see the list of students who have not yet marked attendance.

Acceptance Criteria:

- Dashboard refreshes attendance numbers periodically.
- Low-confidence biometric attempts are highlighted in the UI.
- Pending student list updates as students mark attendance.
- The monitoring view is accessible from the instructor panel during an active session.

---

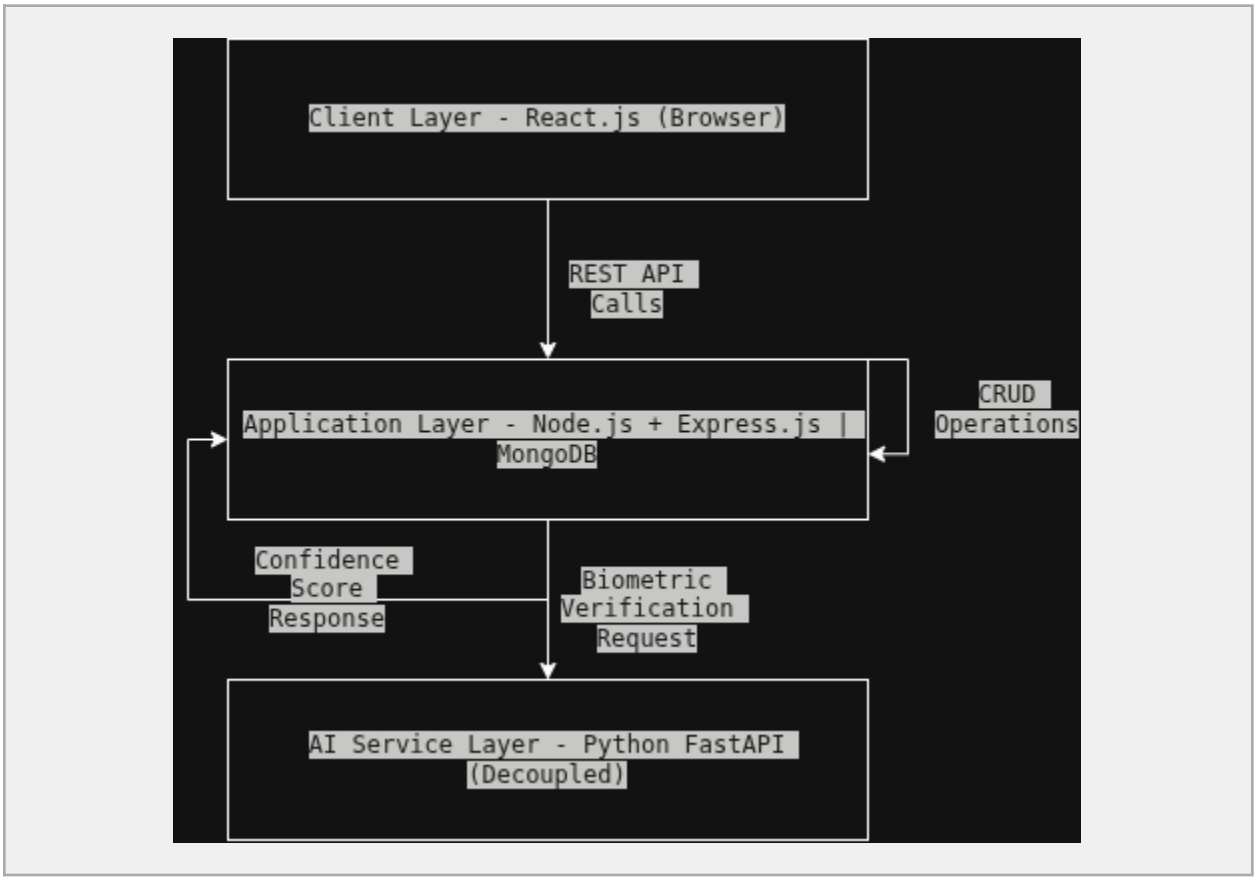
4. Design

4.a Architecture Diagram

Styles Used: Layered Architecture + Microservices (AI service is decoupled).

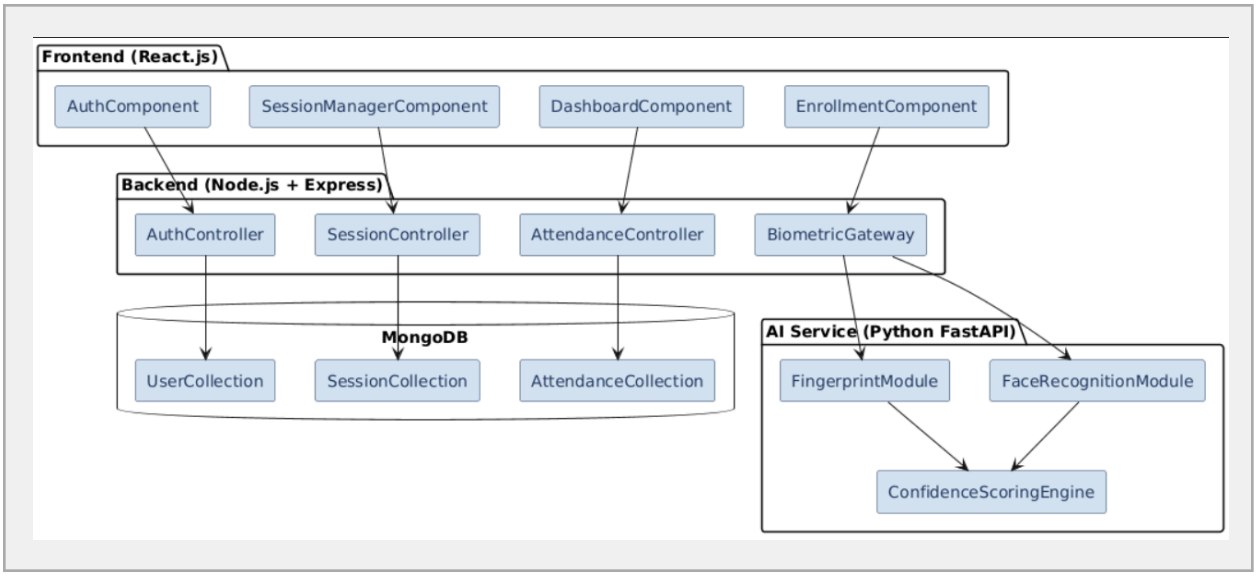
Rationale: Separating the Python AI service from the Node.js backend allows independent scaling, easier model replacement, and clean separation of concerns. The three-tier frontend–backend–database structure keeps the system maintainable.

Description for diagram: Draw three horizontal layers - (1) Client Layer: React.js browser app; (2) Application Layer: Node.js + Express.js REST API + MongoDB; (3) AI Service Layer: Python FastAPI service (decoupled). Show arrows: React → Node API → MongoDB, and Node API → Python FastAPI → returns result.



4.b Component Diagram

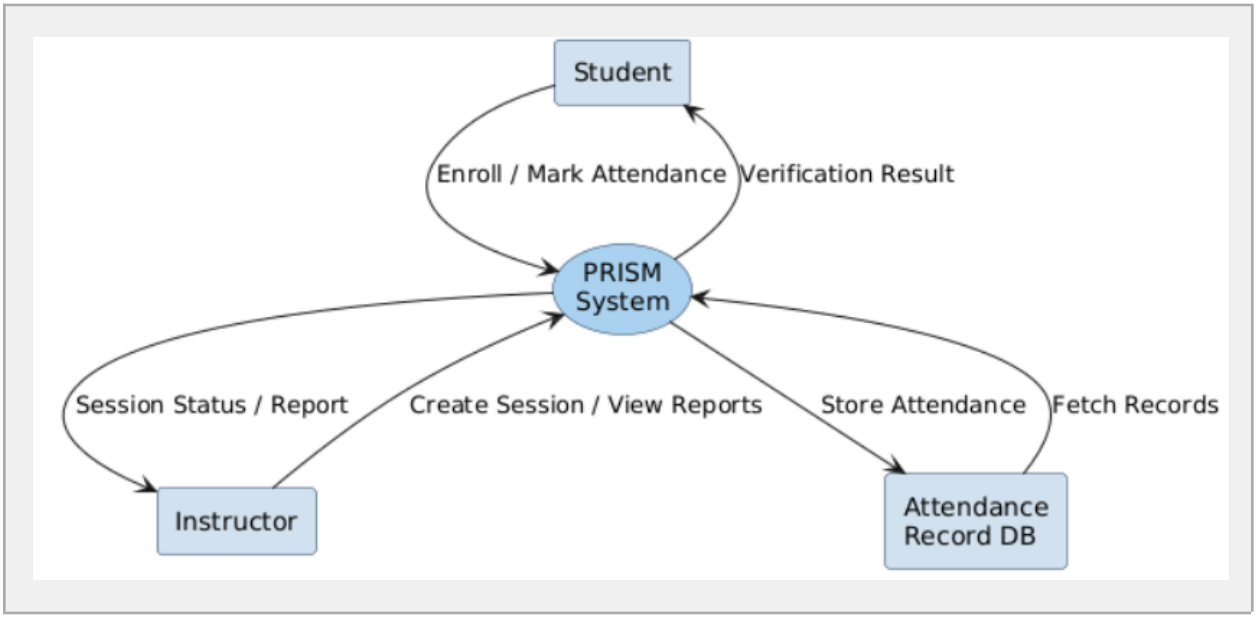
Description for diagram: Show six main components connected by lines - AuthComponent, EnrollmentComponent, SessionManagerComponent, BiometricService (Node side), DashboardComponent, and an external Python AI Service. AuthComponent connects to all others. EnrollmentComponent and BiometricService connect to the Python AI Service. SessionManagerComponent connects to DashboardComponent.



4.c Data Flow Diagram

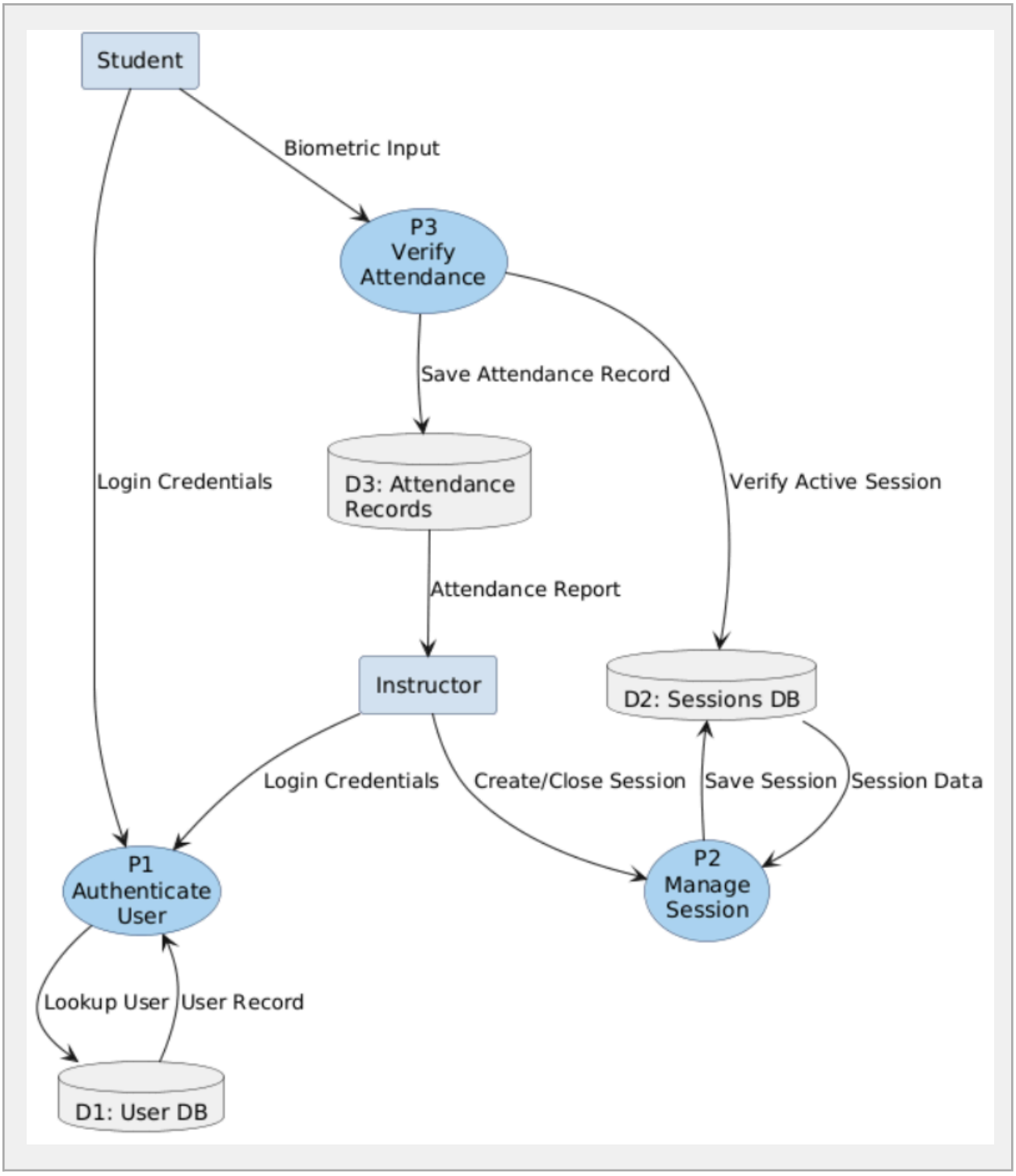
Level 0 - Context DFD

Description: Single process bubble labeled 'PRISM System'. Two external entities: Student and Instructor. Arrows: Student → Enroll/Mark Attendance → PRISM System → Attendance Record → Student; Instructor → Create Session → PRISM System → Session Report → Instructor.



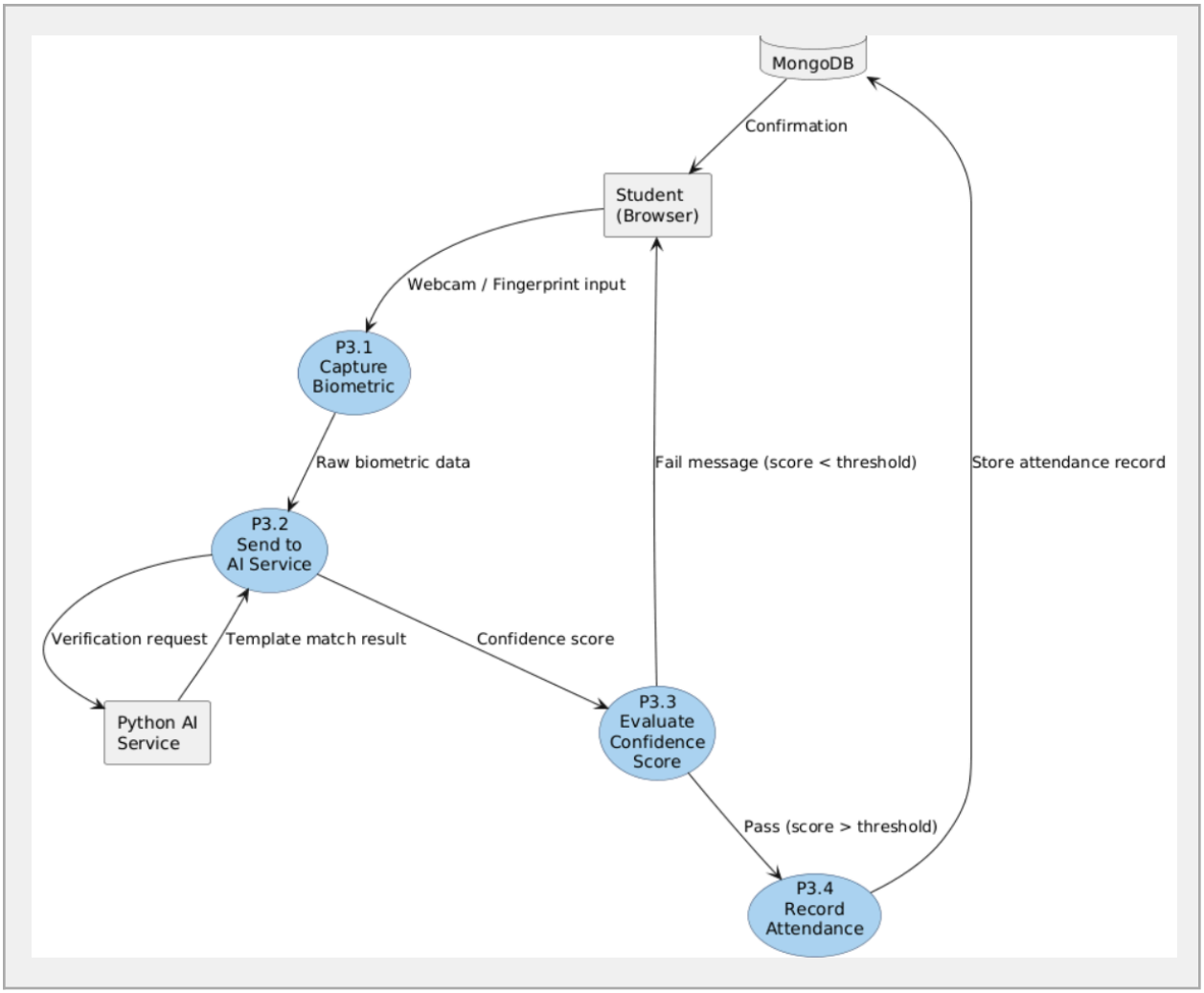
Level 1 - System DFD

Description: Three process bubbles: P1 = Authenticate User, P2 = Manage Session, P3 = Verify Attendance. Data stores: D1 = User DB, D2 = Sessions DB, D3 = Attendance Records. Arrows show data flowing between processes and data stores.



Level 2 - Detailed Verification DFD

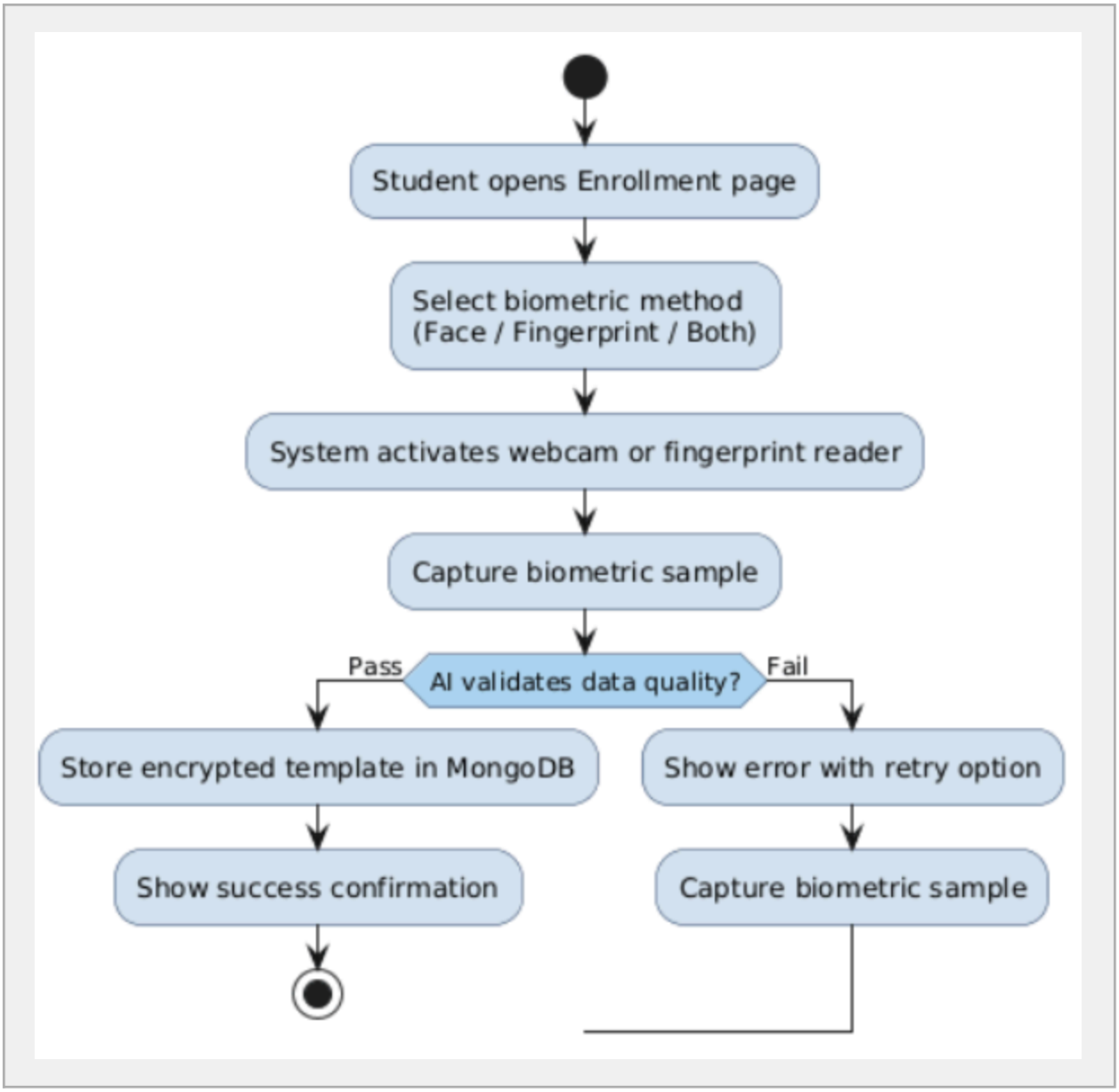
Description: Expand P3. Sub-processes: P3.1 = Capture Biometric, P3.2 = Send to AI Service, P3.3 = Evaluate Confidence Score, P3.4 = Record Attendance. Webcam → P3.1 → P3.2 → Python AI Service → P3.3 → P3.4 → MongoDB.



4.d Activity Diagrams

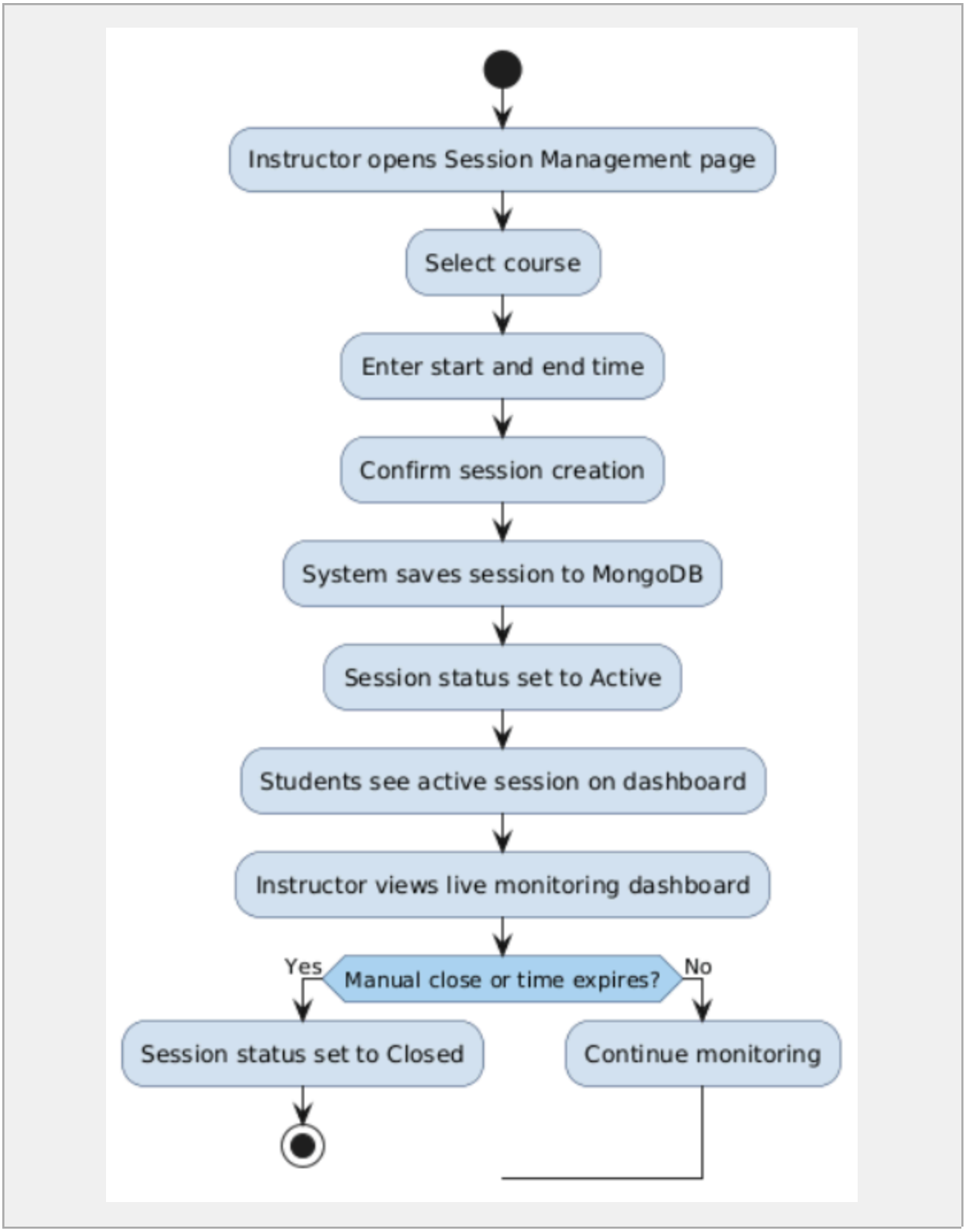
Activity 1: Biometric Enrollment

Description: Start → Student opens Enrollment page → Selects biometric method → System activates webcam/fingerprint reader → Capture attempt → AI validates data quality → [Success] Template saved to DB → Confirmation shown → End. [Failure] → Retry option shown → Loop back to Capture.



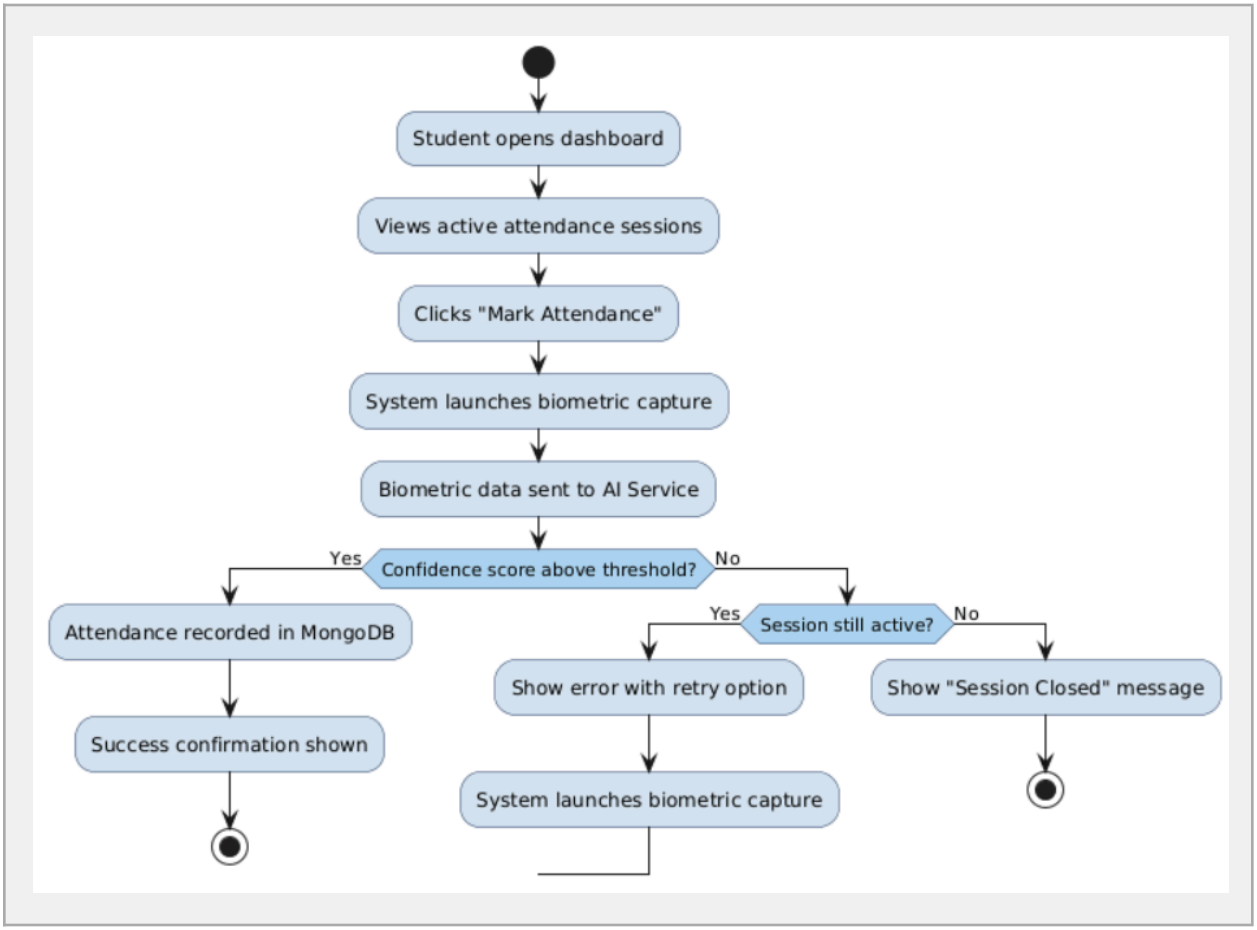
Activity 2: Instructor Creates Session

Description: Start → Instructor opens Session Management page → Selects course → Enters start/end time → Confirms session creation → System saves session to DB → Session status set to Active → Students notified → Instructor views live dashboard → Closes session (manually or auto) → End.



Activity 3: Student Marks Attendance

Description: Start → Student opens dashboard → Views active sessions → Clicks Mark Attendance → System launches biometric capture → Biometric data sent to AI Service → [Verified] Attendance recorded → Confirmation shown → End. [Low Confidence / Failed] → Error shown → Retry or session closed.

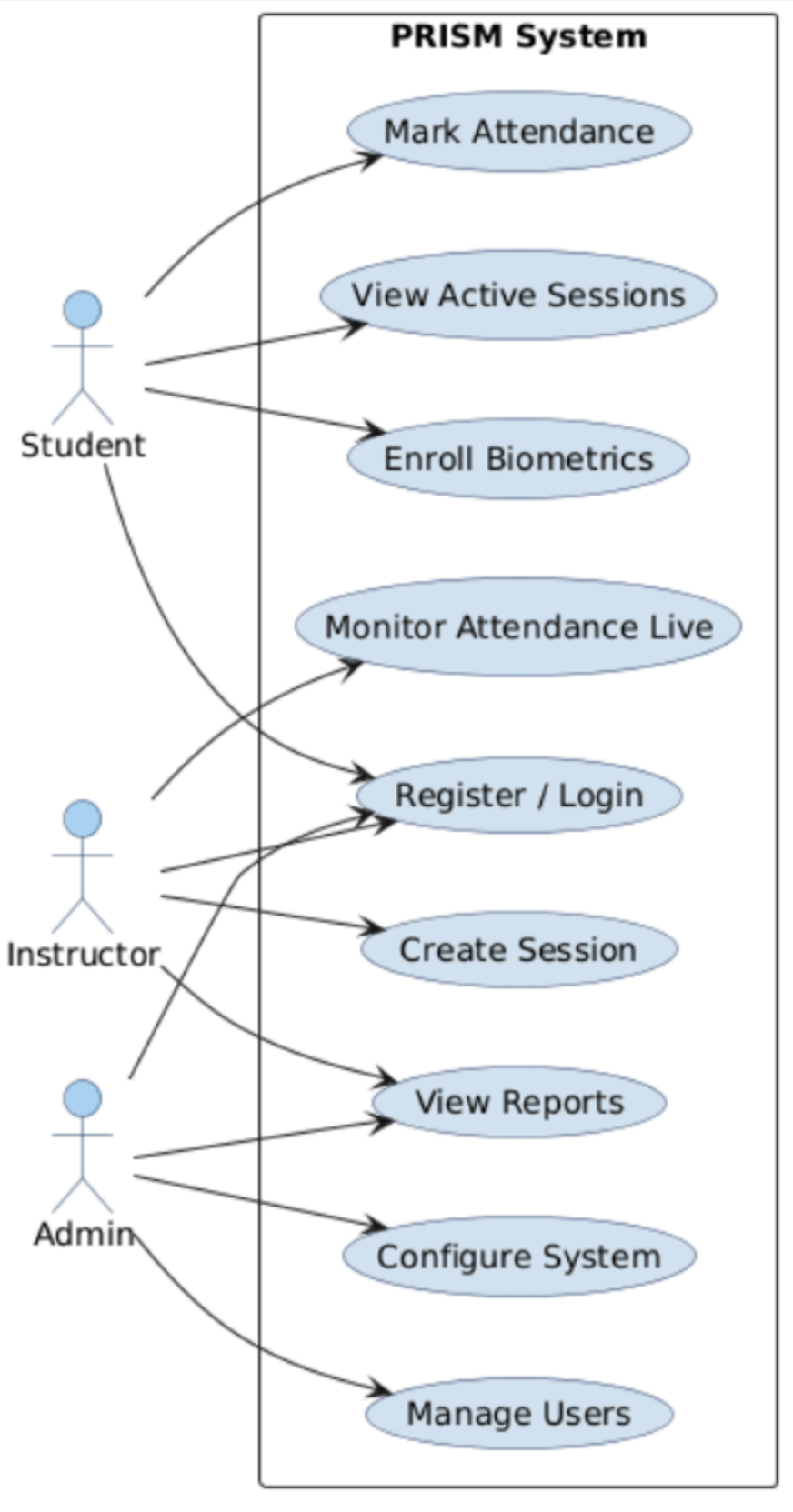


4.e Use Case Diagram

Actors: Student, Instructor, Admin.

Use Cases: Register/Login, Enroll Biometrics, View Active Sessions, Mark Attendance, Create Session, Monitor Attendance Live, View Reports, Manage Users (Admin), Configure System (Admin).

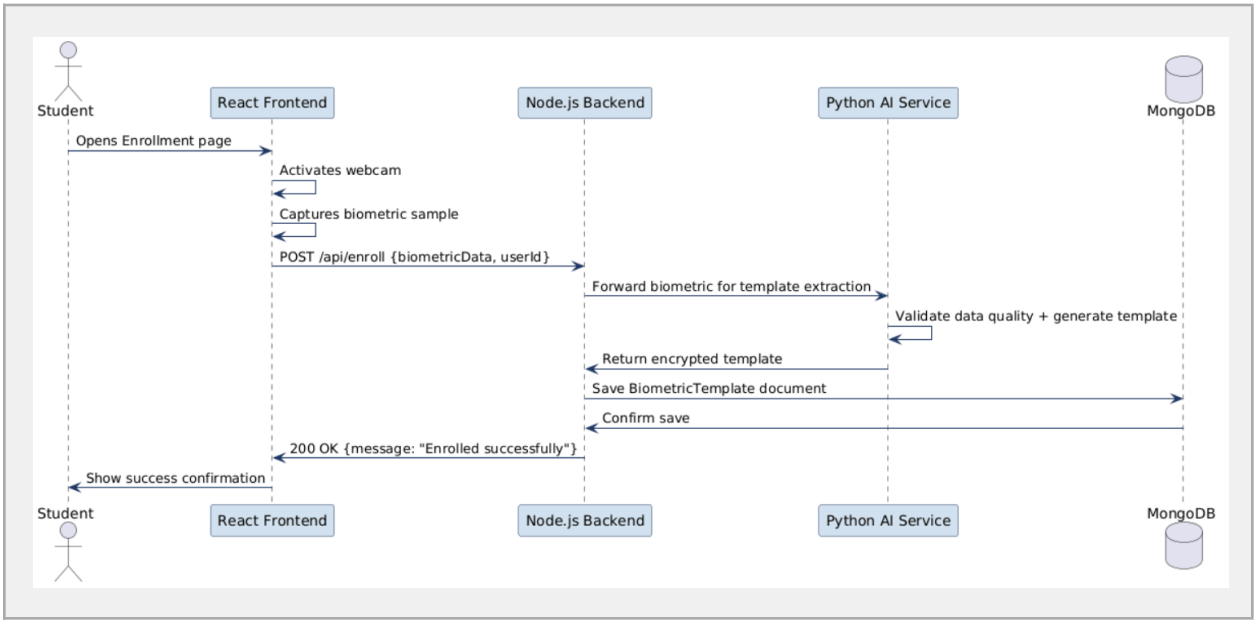
Description for diagram: Draw three stick-figure actors on the sides. Student connects to: Register/Login, Enroll Biometrics, View Active Sessions, Mark Attendance. Instructor connects to: Register/Login, Create Session, Monitor Attendance Live, View Reports. Admin connects to: Manage Users, Configure System, View Reports.


PRISM System
Mark Attendance
View Active Sessions
Enroll Biometrics
Monitor Attendance Live
Register / Login
Create Session
View Reports
Configure System
Manage Users
Student
Instructor
Admin

4.f Sequence Diagrams

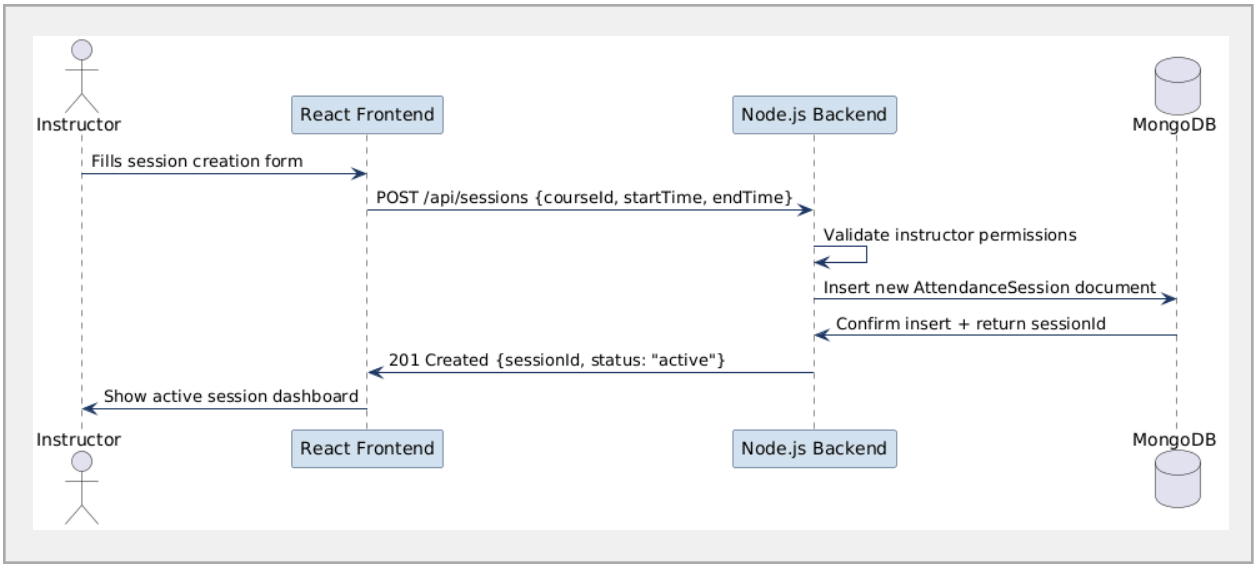
Sequence 1: Biometric Enrollment

Participants: Student (Browser), React Frontend, Node.js Backend, Python AI Service, MongoDB. Flow: Student clicks Enroll → Frontend activates webcam → Captures image → POST /api/enroll → Backend calls Python AI → AI validates template → Backend stores in MongoDB → Returns success → Frontend shows confirmation.



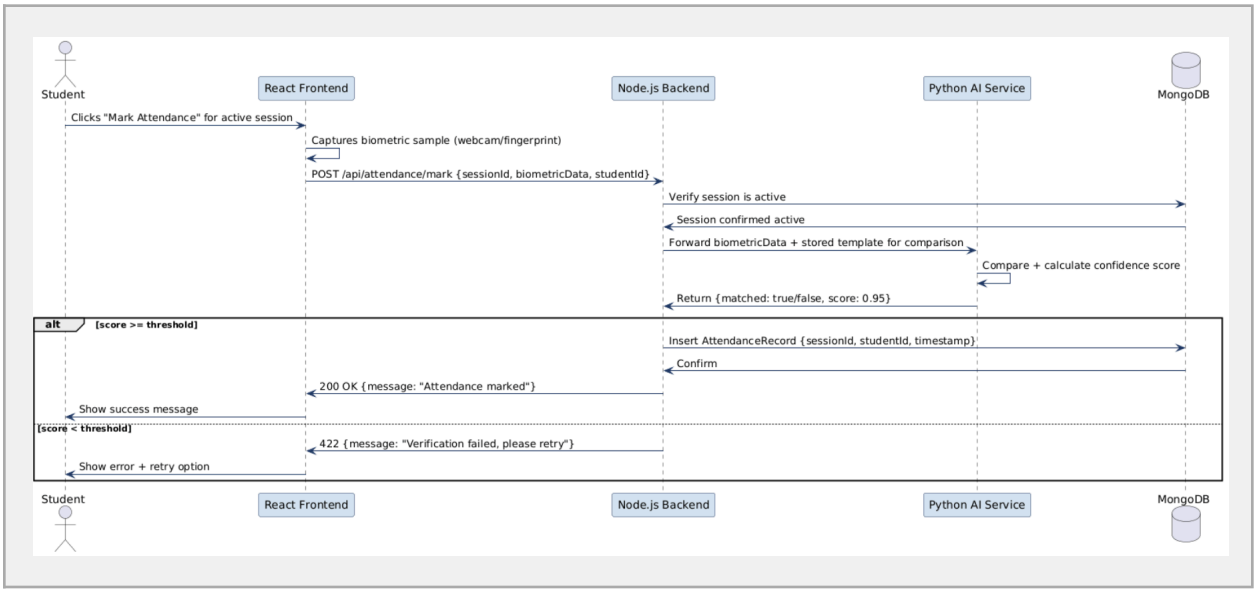
Sequence 2: Instructor Creates Session

Participants: Instructor (Browser), React Frontend, Node.js Backend, MongoDB. Flow: Instructor submits session form → POST /api/sessions → Backend validates data → Saves session to MongoDB → Returns session ID → Frontend shows active session dashboard.



Sequence 3: Student Marks Attendance

Participants: Student (Browser), React Frontend, Node.js Backend, Python AI Service, MongoDB. Flow: Student clicks Mark Attendance → Frontend captures biometric → POST /api/attendance/mark → Backend forwards to Python AI → AI returns confidence score → Backend checks threshold → Records attendance in MongoDB → Returns result → Frontend shows success/failure.

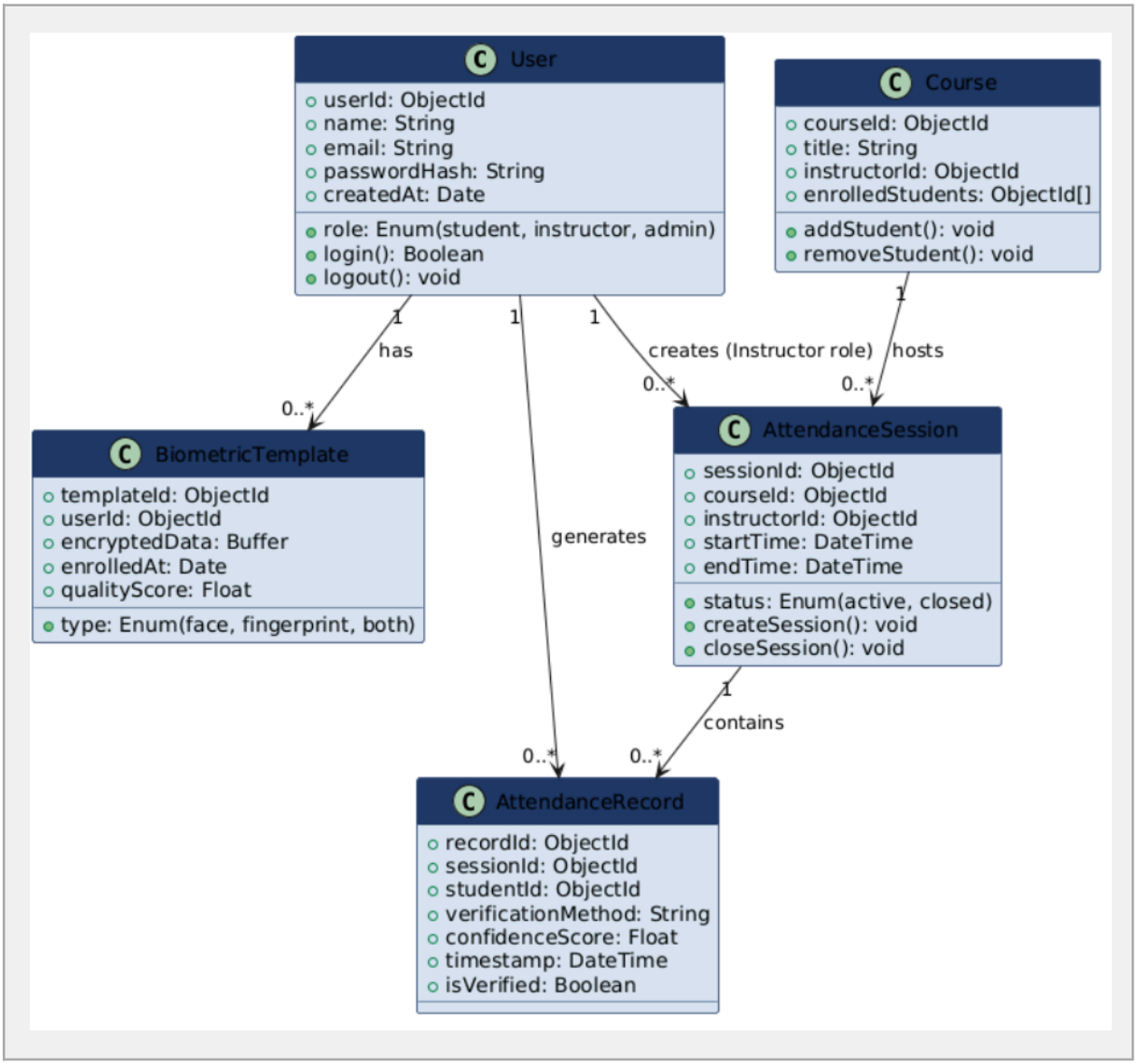


4.g Class Diagram

Classes: User (userId, name, email, role, passwordHash), BiometricTemplate (templateId, userId, type: face/fingerprint, encryptedData, enrolledAt), AttendanceSession (sessionId, courseId, instructorId, startTime, endTime, status), AttendanceRecord (recordId, sessionId,

studentId, verificationMethod, confidenceScore, timestamp), Course (courseId, title, instructorId, enrolledStudents[]).

Relationships: User 1 — 0..* BiometricTemplate; User 1 — 0..* AttendanceRecord; AttendanceSession 1 — 0..* AttendanceRecord; Course 1 — 0..* AttendanceSession.



6. Work Division

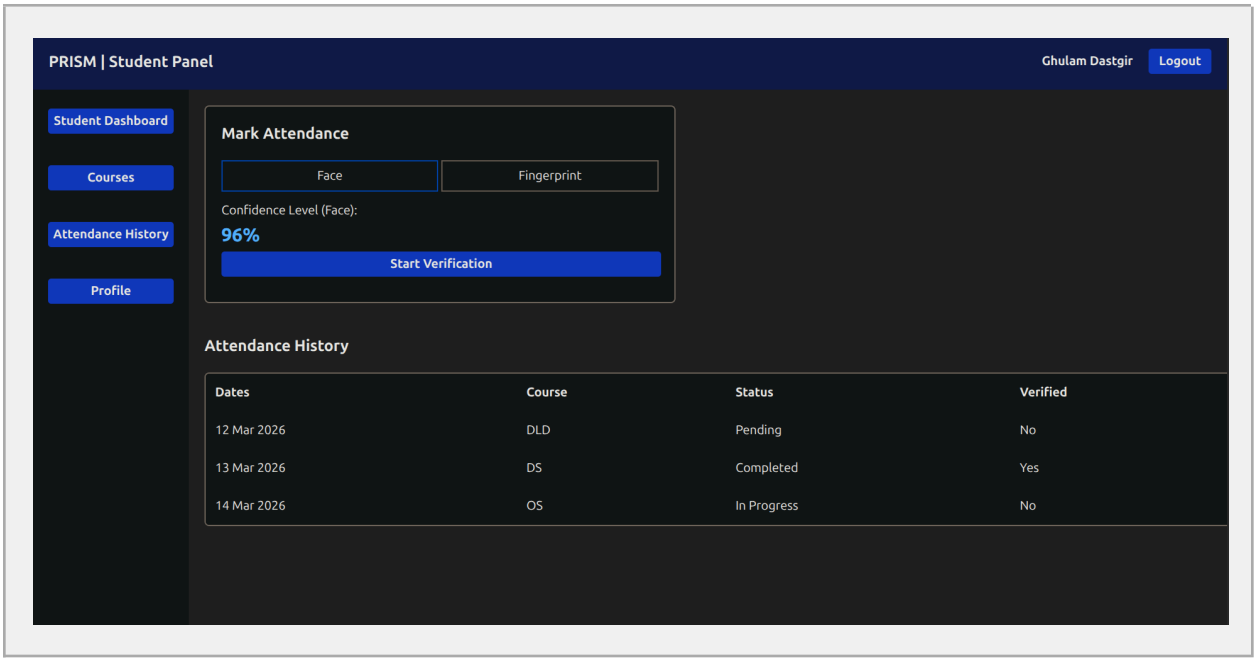
As the sole project member (Areeb Ahmed, 23L-0783), all work was performed individually:

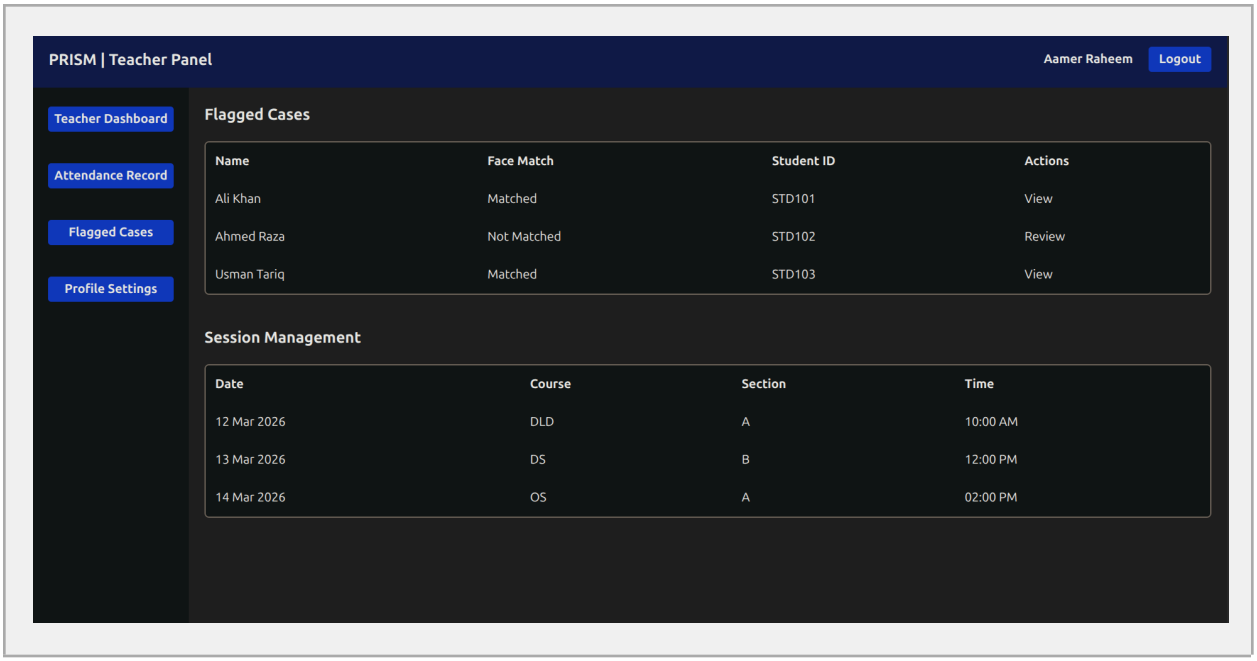
- Designed and implemented the entire frontend - 15 React pages with Tailwind CSS (Login, Signup, Student Dashboard, Teacher Dashboard, Admin Dashboard, and all side-panel tab views).
- Set up the Express.js backend structure: server initialization, route definitions, and MongoDB connection.
- Prepared the Python AI model code for facial recognition and fingerprint validation (not yet connected via FastAPI).
- Created all seven design diagrams (Architecture, Component, DFD x3, Activity x3, Use Case, Sequence x3, Class Diagram).
- Maintained the Trello board throughout Sprint 2 with accurate sprint tracking.
- Authored this complete Deliverable #3 document.

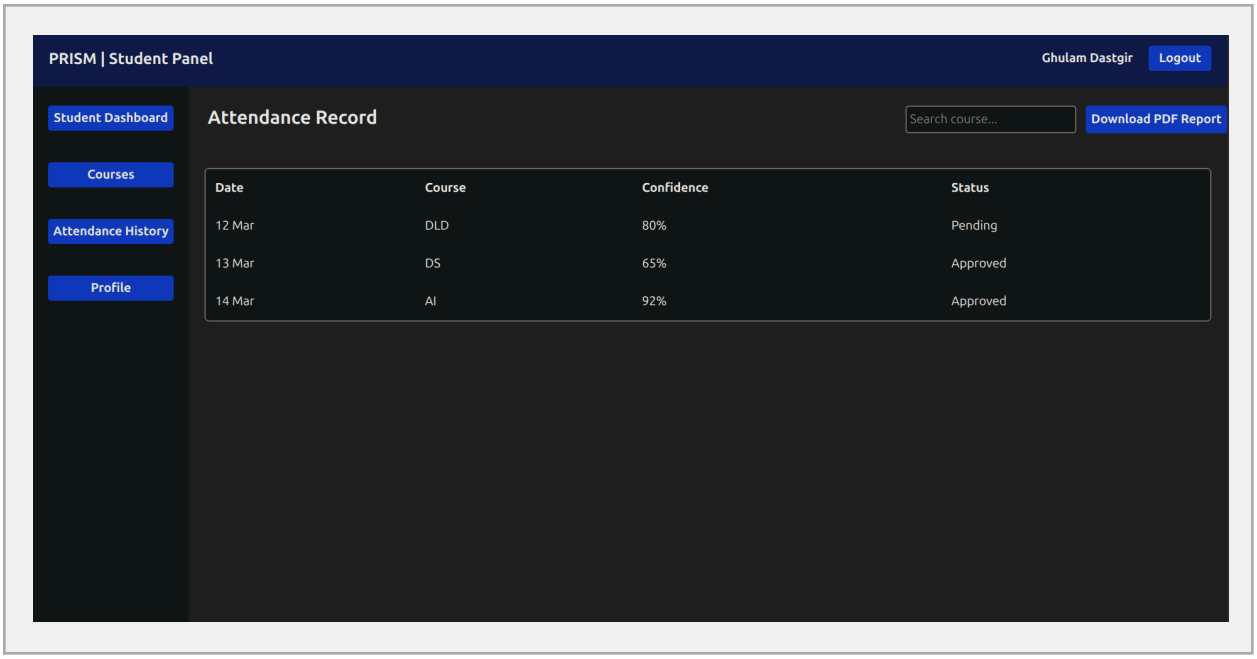
---

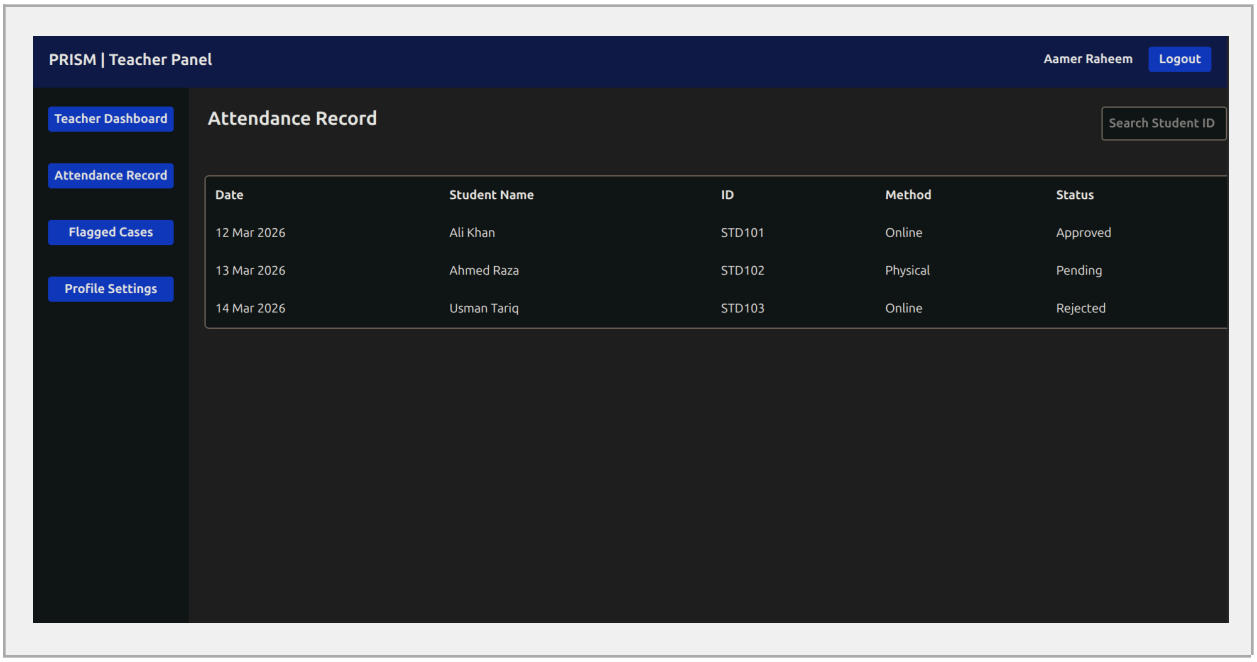
7. Implementation Screenshots

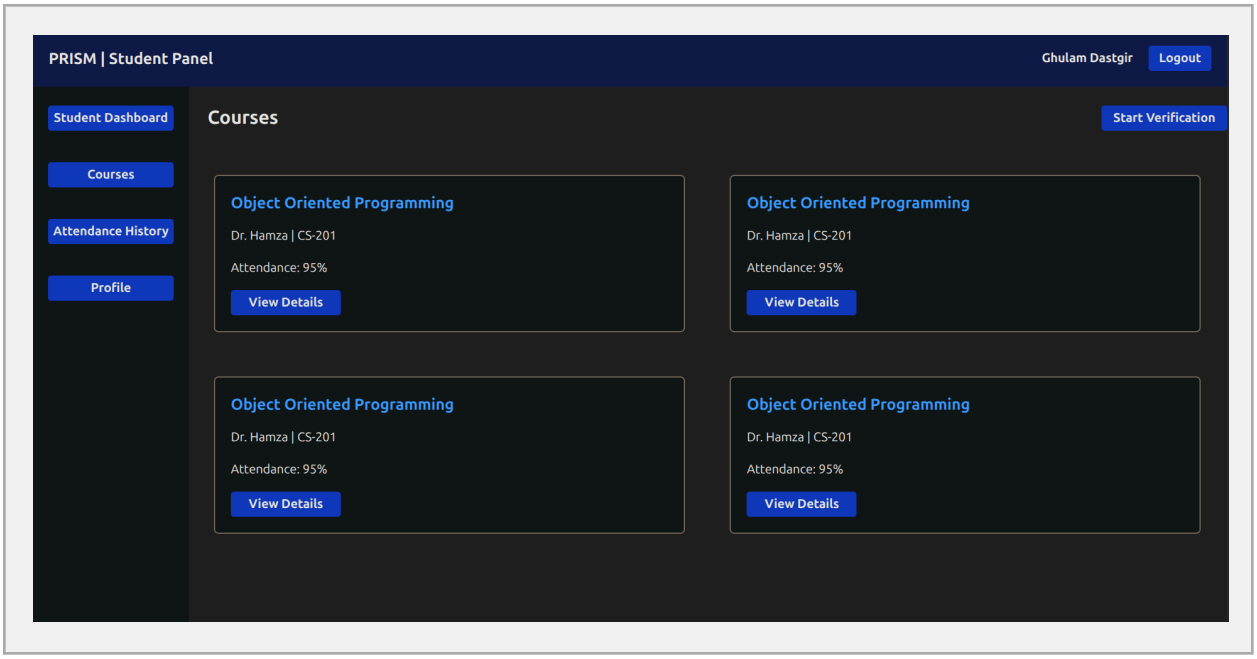
All screenshots below show actual frontend pages implemented during Sprint 2. Frontend is 100% complete. Backend business logic and AI service integration are in progress.






PRISM | Student Panel
Ghulam Dastgir
Logout
Student Dashboard
Courses
Attendance History
Profile
Attendance Record
Search course...
Download PDF Report
Date
Course
Confidence
Status
12 Mar
DLD
80%
Pending
13 Mar
DS
65%
Approved
14 Mar
AI
92%
Approved


PRISM | Teacher Panel
Aamer Raheem
Logout
Teacher Dashboard
Attendance Record
Flagged Cases
Profile Settings
Attendance Record
Search Student ID
Date
Student Name
ID
Method
Status
12 Mar 2026
Ali Khan
STD101
Online
Approved
13 Mar 2026
Ahmed Raza
STD102
Physical
Pending
14 Mar 2026
Usman Tariq
STD103
Online
Rejected



End of Document